

Utilizing Conditional Cells and Ports

coreBuilder Application Note

Mark Noll, Synopsys Design Reuse Group

July 2000

Abstract

Support for configuration is one of the key features of coreBuilder. In addition to the standard configuration via parameters supported by VHDL and Verilog, coreBuilder supports the conditional inclusion of both cells and ports within a design. This application note describes how to take advantage of this functionality in both VHDL and Verilog.

Introduction

Configuration of RTL designs is most commonly handled via parameters (generics and constants in VHDL, or params and 'defines in Verilog). However it is also possible to configure VHDL based designs using the 'generate' statement to conditionally include cells and other functionality. Also, Verilog supports the concept of a pre-processor that can be used to conditionally include and exclude arbitrary sections of code. In order to be able to properly synthesize a design, coreBuilder must have a model that accurately represents the design hierarchy. This means that the tool must understand what is being conditionally included and excluded from the design. The coreBuilder tool currently supports the conditional inclusion (or exclusion) of cell instantiations and port definitions. This is handled automatically for cells in VHDL since the parser sees and understands the 'generate' statements in that language. However, for ports in VHDL and cells and ports in Verilog, extra work is required to support conditional inclusion. This Application Note describes what is required to utilize these features.

Conditional Inclusion of Cells

The following applies to Verilog only since this is handled automatically for VHDL.

In order to support conditional inclusion of a cell in a Verilog design, two things must be done. First, an attribute must be applied to the cell to indicate to coreBuilder when it should be included. Second, a text substitution pragma must be added to the HDL code so that the text instantiating the cell is included or excluded based on the configuration of the design. The attribute to set on the cell is called `Generatelf`. Its value should be a parameter expression that evaluates to true when the cell should be included in the design. The attribute can be applied via reuse-pragmas in the HDL code (the preferred method) or via a call to `set_cell_attribute` in a

TCL intent file. An example is shown below. Note that they also include the mandatory text-substitution pragmas.

```
// Instance U2 is only included when constant 'W' has the value 3.
// The constant is defined in myconstants.inc.
#include myconstants.inc
module top (in1, in2, out1, out2);
    input in1;
    input in2;
    output out1;
    output out2;

    U1 mysubmodule(in1, out1);
    // reuse-pragma attr GenerateIf @W==3
    // reuse-pragma startSub [parameter_conditional_text %subText @W==3]
    U2 mysubmodule(in2, out2);
endmodule;
```

The first reuse-pragma line sets the GenerateIf attribute on U2 since it is the next object defined in the Verilog file. The implication is that U2 appears only when W has the value 3. The second reuse-pragma line defines a text substitution. This form of substitution pragma replaces the next line a file with the result of the given TCL command. In this case, the built-in parameter_conditional_text command is being used. This command will return the cell instantiation line if and only if W is 3. The block format of the substitution pragma can also be used if the cell instantiation takes up more than one line, as shown below.

```
// reuse-pragma startSub U2 [parameter_conditional_text %subText @W==3]
U2 mysubmodule(in2,
               in3);
// reuse-pragma endSub U2
```

Note that the token 'U2' appearing in both the startSub and endSub pragmas can be any string, provided that they match, as this defines the block of text to be replaced.

To test this functionality, package the core, install it, and then create an integration workspace using coreConsultant. Then complete the SpecifyConfiguration activity once with W set to 3 and once without. A review of the configured source code should show that U2 is present in the first case and not in the second.

Conditional Inclusion of Ports

The conditional inclusion of ports works in exactly the same way as it does for cells. The `GenerateIf` attribute must be applied to the port to indicate to the data model when it should be present, and a text-substitution pragma must be provided to enable the configured code to be written out properly based on the configuration. A Verilog example is shown below.

```
// module top contains 1 input and 1 output, unless
// constant 'W' has value 3. In that case, there
// is an extra input 'in2', and extra output 'out2'.
`include myconstants.inc
module top (in1
    // reuse-pragma startSub [parameter_conditional_text -design top %subText @W==3]
    ,in2
    ,out1
    // reuse-pragma startSub [parameter_conditional_text -design top %subText @W==3]
    ,out2
);
    input in1;
    // reuse-pragma attr GenerateIf @W==3
    // reuse-pragma startSub [parameter_conditional_text -design top %subText @W==3]
    input in2;
    output out1;
    // reuse-pragma attr GenerateIf @W==3
    // reuse-pragma startSub [parameter_conditional_text -design top %subText @W==3]
    output out2;
endmodule;
```

Note that the substitution pragmas must be provided for **both** the port declaration within the module statement and the secondary definition which indicates the port type. In both cases, the block substitution mechanism could be used, though it would require re-ordering the ports to use it within the module declaration. For that reason, the example below shows this only within the port type declaration section.

```
input in1;
output out1;
// reuse-pragma attr GenerateIf @W==3
// reuse-pragma startSub W3 [parameter_conditional_text -design top %subText @W==3]
input in2;
// reuse-pragma attr GenerateIf @W==3
output out2;
// reuse-pragma endSub W3
```

A complete Verilog example containing both conditional cells and ports, and a VHDL example containing conditional ports are given in the next section.

Complete Examples

```
// module top contains 1 input and 1 output, unless
```

Utilizing Conditional Cells and Ports

```

// constant 'W' has value 3. In that case, there
// is an extra input 'in2', and extra output 'out2' and an
// extra cell instantiation 'U2'.
`include myconstants.inc
module top (in1
    // reuse-pragma startSub [parameter_conditional_text -design top %subText @W==3]
    ,in2
    ,out1
    // reuse-pragma startSub [parameter_conditional_text -design top %subText @W==3]
    ,out2
    );
input in1;
// reuse-pragma attr GenerateIf @W==3
// reuse-pragma startSub [parameter_conditional_text -design top %subText @W==3]
input in2;
output out1;
// reuse-pragma attr GenerateIf @W==3
// reuse-pragma startSub [parameter_conditional_text -design top %subText @W==3]
output out2;

U1 mysubmodule(in1, out1);
// reuse-pragma attr GenerateIf @W==3
// reuse-pragma startSub [parameter_conditional_text %subText @W==3]
U2 mysubmodule(in2, out2);
endmodule;

```

Example 1: Verilog

```

-- entity top contains 1 input and 1 output, unless
-- constant 'W' has value 3. In that case, there
-- is an extra input 'in2', and extra output 'out2' and an
-- extra cell instantiation 'U2'.
package myconstantpkg is
    constant W : natural range 0 to 5 := 3;
end myconstantpkg;
library work;
use work.myconstantpkg.all;
entity top is
    port(in1: in bit;
        -- reuse-pragma attr GenerateIf @W==3
        -- reuse-pragma startSub [parameter_conditional_text -design top %subText @W==3]
        in2: in bit;
        out1: out bit;
        -- reuse-pragma attr GenerateIf @W==3
        -- reuse-pragma startSub [parameter_conditional_text -design top %subText @W==3]
        out2: out bit
    );
end top;

architecture structural of top is
    component
        mysubmodule port(A : in std_logic; Z : out std_logic);
    end component;
begin
    U1: mysubmodule port map(A => in1, Z => out1);
    G1: if (W = 3) generate
        U2: mysubmodule port map(A => in2, Z => out2);
    end generate G1;
end structural;

```

Example 2: VHDL